

Mallow Technologies

Associate Team Lead — Laravel (PHP)

Mini Task

Thank you for taking the time to work on this assignment. Please read the full brief before you start. If anything is ambiguous, use your best judgment, make a reasonable assumption, and document it in your README — we're as interested in how you handle ambiguity as in the result. This task also gives us a look at how you'd hand work off to a team — a follow-up conversation will dig into the trade-offs and judgment calls behind what you built, not just whether it works.

Brief: Subscription Billing & Usage-Metering System

Design and build a small backend that meters customer usage against a subscription plan and produces billing, for a multi-tenant SaaS-style scenario.

Scope

- Merchants (tenants), each defining one or more Plans (name, base price, billing cycle, included usage units, overage rate per unit).
- Customers subscribe to a merchant's plan.
- Usage Events are recorded per customer per day (e.g. API calls) — assume high write volume.
- At cycle end, the system generates an invoice: base price + overage beyond the included allowance, with proration if the subscription started mid-cycle.

Functional Requirements

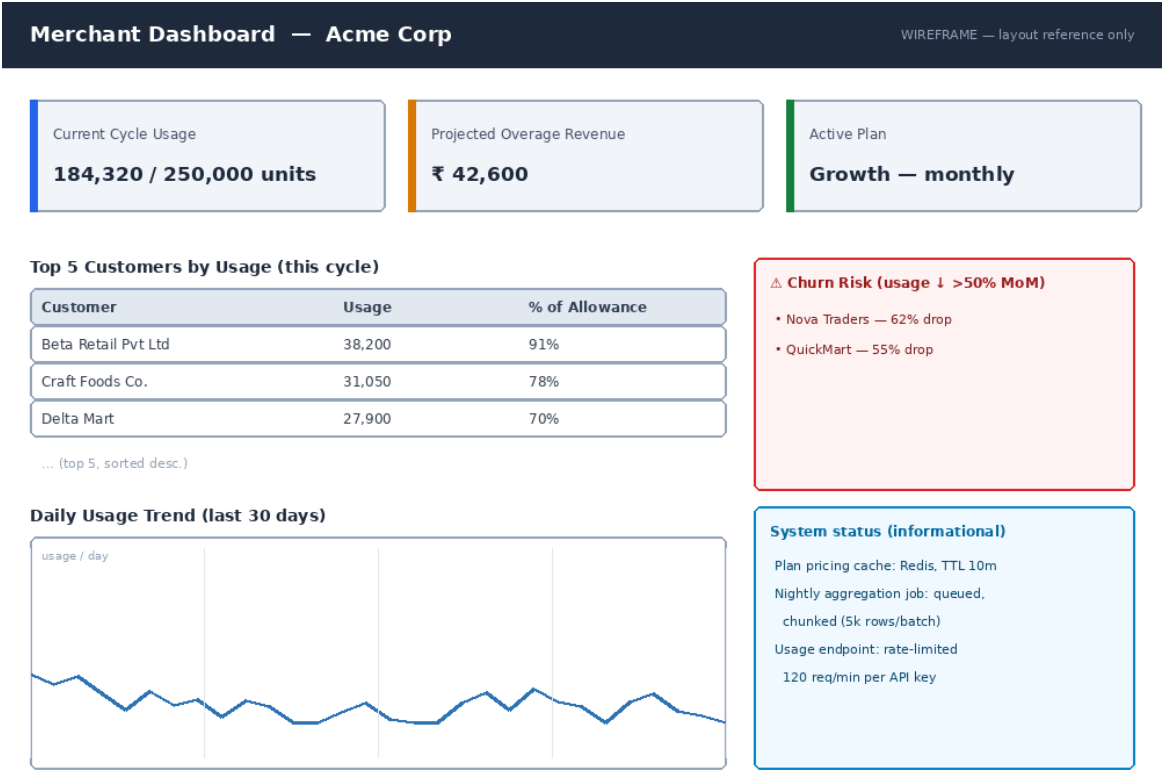
1. Design a normalized, indexed schema. Document explicitly how you'd expect it to hold up with 50L+ usage-event rows and note any denormalization/partitioning you'd consider.
2. POST /usage endpoint to record a usage event. It should be safe to call at high throughput and idempotent (a retried request must not double-count).
3. A queued, chunked job that aggregates a customer's daily usage and, at cycle end, generates the invoice with correct proration and overage math.
4. Cache plan/pricing lookups (Redis or array cache is fine for the exercise) and document your invalidation strategy.
5. GET /merchants/{id}/dashboard returning: top 5 customers by usage this month, projected overage revenue for the current cycle, and customers whose usage dropped >50% month-over-month (churn risk).
6. Basic rate-limiting on the usage endpoint.
7. Tests for the aggregation and billing calculation, including proration and overage edge cases.
8. Handle a customer upgrading or downgrading their plan mid-cycle: usage recorded before the change must be billed at the original plan's rate, and usage after the change at the new plan's rate, with proration reflecting both segments correctly.
9. **Code review exercise.** Below is a first draft of the usage-recording endpoint, opened as a pull request by a mid-level engineer on your team. It's in your queue to review before anything ships. Handle it however you think is right.

```
public function store(Request $request)
{
    $customer = Customer::find($request->customer_id);
    $usage = new UsageEvent();
    $usage->customer_id = $customer->id;
    $usage->units = $request->units;
    $usage->usage_date = $request->usage_date;
    $usage->save();
    return response()->json(['status' => 'ok']);
}
```

10. **Rollout & monitoring note.** Add a short paragraph to your README (a few sentences, not a runbook): how would you roll this out with minimal risk, and what's the one thing you'd want an on-call engineer to see first if usage recording started silently failing in production?

Suggested UI Reference (Wireframe)

This is a low-fidelity layout reference for the dashboard — it shows the expected panels and data points so you can visualize scope before you start. Visual styling is entirely up to you; we're evaluating the schema, aggregation and caching decisions behind it, not the UI polish.



AI-assisted development (Copilot, Cursor, Claude, etc.) is welcome, no restriction to use AI.

What to Submit

- Codebase: a GitHub repo (private, shared with us) containing the full application.
- README.md: architecture summary, setup/run instructions, and a clear write-up of assumptions and design decisions made. If any requirement is ambiguous, make a reasonable assumption, proceed, and document it here — don't stop and wait to ask. Also include a short "Handing this off" section — the 2–3 things you'd flag to a mid-level engineer picking this up next, and anywhere you knowingly cut a corner under the time-box.
- Prompt Log: if you used AI-assisted tools (Copilot/Cursor/Claude etc.), include screenshots of the actual prompts you used (from your chat/IDE panel) — so we can see exactly what was asked.
- Screen Recording: a short (5–10 min) video walkthrough of the working application and its architecture, narrated by you — walk us through your key decisions. A silent screen capture isn't sufficient.

Timebox & Submission

- Timebox: 3 days elapsed time from receiving this brief.
- Please share your GitHub repo link, README, prompt screenshots (if applicable) and narrated screen recording with [recruiter/HR email] once ready.
- If anything in this brief is unclear, think it through, make the call you believe is most reasonable, and document your reasoning in the README.md — we're evaluating your judgment under ambiguity, not whether you guessed our exact intent.